

Parsity

AI Engineering Program

Curriculum Overview

Program Summary

The Parsity AI Engineering Program prepares working software engineers to design, build, and deploy production-grade AI applications. The curriculum centers on Retrieval-Augmented Generation (RAG), agentic systems, and the engineering practices required to operate these systems reliably in real environments. Coursework is built around a single end-to-end TypeScript and Next.js codebase that learners extend across the program, culminating in an individual capstone project.

Instruction emphasizes engineering fundamentals over framework familiarity. Learners implement core mechanics, including vector similarity, chunking, retrieval, reranking, structured outputs, and evaluation, before adopting higher-level abstractions. The program is cohort-based, project-driven, and instructor-mentored, and is delivered in connection with a U.S. Department of Labor Registered Apprenticeship.

Learning Outcomes

Upon successful completion of the program, learners will be able to:

- Explain the architecture and tradeoffs of retrieval-augmented generation systems and articulate when RAG, fine-tuning, or prompting is the appropriate technique.
- Implement vector embeddings, similarity search, and hybrid (sparse plus dense) retrieval pipelines using industry-standard tooling.
- Design and build agentic systems that route between specialized capabilities using structured outputs and validated schemas.
- Apply software engineering practices, including testing, observability, and graceful degradation, to non-deterministic AI systems.
- Evaluate AI systems using both deterministic tests and LLM-as-judge methodologies.
- Communicate technical decisions, tradeoffs, and system designs in professional contexts.
- Independently scope, build, and present a production-quality AI application addressing a defined real-world use case.

Prerequisites

Working proficiency in JavaScript or TypeScript; familiarity with Git, command-line tooling, and REST APIs; comfort building and running a Node.js application.

Technology Stack

TypeScript, Next.js, Node.js, Pinecone (vector database), OpenAI API, Zod (schema validation), LangSmith (observability). All work is performed in a shared instructional repository with reference and student-exercise branches.

Course Structure

The program is organized into sixteen instructional modules grouped into five thematic units. Modules build sequentially; later modules assume mastery of earlier material.

Unit I: Foundations of Retrieval-Augmented Generation

Module 1. Introduction to Retrieval-Augmented Generation. Motivation for retrieval augmentation, representative real-world applications, and an architectural overview of a complete RAG system.

Module 2. Vector Mathematics and Embeddings. Vector representations, cosine similarity implemented directly, and the geometric properties of embedding spaces.

Module 3. Vector Database Integration with Pinecone. Client configuration, index provisioning, and upsert and query operations against real embedding data.

Module 4. Document Processing and Chunking Strategies. Document ingestion, web scraping fundamentals, and the effect of chunk size, overlap, and boundary heuristics on retrieval quality.

Module 5. Document Upload Pipeline. An end-to-end ingestion pipeline: document loader, Next.js API route, and query functionality against the populated index.

Unit II: Agent Architecture and Specialization

Module 6. Fine-Tuning: Theory and Application. Comparative treatment of fine-tuning and retrieval augmentation, including execution of a representative fine-tuning job.

Module 7. Agent Architecture and Structured Outputs. Router design, schema-validated structured outputs using Zod, and graceful degradation patterns including fallbacks and retry logic.

Module 8. Specialized Agent: LinkedIn Content. Prompt engineering for a domain-specific agent, examining task framing, constraint specification, and example selection.

Module 9. RAG Agent with Hybrid Search and Reranking. The program's core RAG agent, integrating a reranking stage and hybrid search across sparse and dense representations.

Unit III: User Interface and Production Operations

Module 10. Conversational Interface and Streaming. Streaming response handling, conversation context management, and interaction patterns for usable AI interfaces.

Module 11. Observability with LangSmith. Distributed tracing, debugging agent behavior across nested calls, and monitoring practices for production deployment.

Module 12. Testing AI Agents. Deterministic testing of agents and the LLM-as-judge evaluation pattern, compared against assertion-based testing.

Unit IV: Advanced Techniques and Structured Retrieval

Module 13. Tool Calling and Agent Capabilities. The tool-calling contract, designing well-bounded tools, and the boundary between deterministic code and model-mediated behavior.

Module 14. Structured Data Retrieval: SQL Agents. Text-to-SQL patterns, query validation, and safety considerations for model-generated database queries.

Unit V: Capstone and Professional Preparation

Module 15. Capstone Project. An original RAG application addressing a real-world use case, with instructor checkpoints, documentation, and a presented demonstration.

Module 16. Professional Preparation. Constructing a project narrative, defending technical decisions, RAG system-design questions, and live mock practice with instructor feedback.

Assessment

Learners complete five graded deliverables, each evaluated against a published rubric covering correctness, engineering quality, and the ability to articulate design decisions.

#	Assignment	Aligned Module
1	Document Upload Pipeline	Module 5
2	Selector Agent with Structured Outputs	Module 7
3	RAG Agent with Reranking	Module 9
4	Text-to-SQL Agent	Module 14
5	Capstone Project	Module 15

Contact

Parsity | parsity.io | brian@parsity.io